

PyQCU 核心算法深剖（补充）：多重网格 V-cycle

15 步全流程重现（代码—物理交叉框架 A）

opencode pure

2026-08-18

摘要

本报告作为 `pure_pyqcu_20260818` 的补充，对 PyQCU 最强竞争力——多重网格 (MultiGrid) V-cycle 求解器——按「分析思路框架」A（代码/软件工作）15 步做全流程深剖。证据均来自当前 HEAD (`a456932`) 逐行 `read/gzip` 核实。核心结论：V-cycle 以近零空间向量 (null space) 经 Galerkin 投影 $A_c = P^T D_{fine} P$ 构造粗算子，递归消除低频误差；细层用 C++ CUDA 后端 BiCGStab 平滑 (5-stream 架构)，粗层纯 Python；粗网格校正后强制重置 BiCGStab 全部状态 (R3 fix) 并支持 breakdown 自动重启 (R2 演进)。实测收敛数据引自 `pyqcu/solver/AGENTS.md`。

目录

1 候选归属与性质判定

本报告聚焦 `pure_pyqcu_20260818` 清单中的 C4（多重网格 V-cycle，深挖）。项目性质为**代码—物理交叉**：V-cycle 的每个代码符号 (`lonv_list`、`restrict/prolong`、`cycle(level)`、`v_cycle`) 均有唯一物理对象（低频模态基 P 、限制/延拓、 D_l 层算子、递归粗层），故以 A 框架为主体、B 元素（物理图像）并入各步。

2 工作全流程重现（A 代码工作框架）

2.1 A1 目的与前期准备

为什么存在：求解格点 QCD 狄拉克方程 $D\psi = \eta$ 时，细层 Dirac 算子 D 的条件数随体积 $\propto V$ ，普通 Krylov 法（如 BiCGStab）在红外（低频）模态上收敛极慢；多重网格用粗网格显式消除低频误差，将迭代次数与体积解耦（`pyqcu/solver/AGENTS.md`「multigrid 类」）。**前置条件**：规范场 U 、Clover 项、 κ/u_0 、近零空间向量；生产路径需先 `build.sh+install.sh` 构建 `libqcu.so` 与 Cython 扩展（`AGENTS.md:10`）。

2.2 A2 输入

输入	类型	参考源
U （规范场）	[3,3,4,X,Y,Z,T]	_multigrid.py:241
clover_ee/oo	块对角 Clover	_multigrid.py:243-244
κ, u_0	标量	AGENTS.md 构造参数
lonv（近零空间）	迭代代生成	_multigrid.py:273-278
b （右端）	费米子场	_multigrid.py:340,524

2.3 A3 输出（应是什么）

解 x （费米子场， ψ ），以及 `convergence_history`（每轮残差，用于 `adaptive` 停滞判定与绘图，_multigrid.py:368,434,484）。正确性判据：细层残差降至 `tol`（默认绝对容差，_multigrid.py:369）。

2.4 A4 整体框架

多层递归：层 l 上先 BiCGStab 平滑，再做粗网格校正——将残差限制到层 $l+1$ 、递归 `cycle(l+1)` 求粗解、延拓回细层校正；最细层（ $l = 0$ ）走 C++ CUDA 后端（`with_cuda_qcu=True`），层 1 走 C++ 粗网格 `dslash`，层 2+ 纯 Python `einsum`（`pyqcu/solver/AGENTS.md`「执行层」）。递归入口 _multigrid.py:455 `self.cycle(level=level+1)`。

2.5 A5 关键细节

- **Galerkin 粗算子**：`operator()` 用 `prolong+matvec+restrict` 单格点探测 $P^T D_{fine} P$ ，写入 `sitting.M` 与 `hopping.M_plus/minus_list`（_operator.py:157-226）。
- **R3 fix**：粗校正后 `x=x+e_fine` 使残差改变，必须重置 `r_tilde,p,v,s,t,rho,rho_prev,alpha,omega`（_multigrid.py:465-482）。
- **自动重启**：`breakdown(rho/rtv/tts<1e-20 或非有限)` 时保留 `x/r` 重置状态后继续（_multigrid.py:392-430）。
- **5-stream**（C++ 端）：`main/_a/_b/_c/_d` 并行 BiCGStab 子步骤（`lattice_clover_multigrid.h:20-31`）。

2.6 A6 任务如何正确执行

典型调用链（生产路径）：`multigrid.init()`（构建 `null` 空间与粗算子） $\rightarrow$ `multigrid.solve(b)` $\rightarrow$ `cycle(0)` 递归 $\rightarrow$ `multigrid.end()` 释放 `scratch`（_multigrid.py:204,523,290）。C++ 后端每次调用间必须 `params[_SET_INDEX_]+=1`（_multigrid.py:262），否则 `scratch` 冲突。

2.7 A7 实际执行情况

未验证（本静态分析会话未构建 `libqcu.so` / 无 GPU 运行）。以下为仓库既有实测记录（`pyqcu/solver/AGENTS.md`「CUDA 混合路径同步与 `breakdown` 重启」「多线程实例并行」）：

要点

文档化实测：8x8x8x8 1L 残差 8.7e-7; 8x8x8x16 2L 残差 7.8e-7 (自动重启后收敛不受影响); 2 线程 8x8x8x8 1L 并行各 6.3e-7/9.5e-7 均收敛; 3 实例循环显式 `end()+del+gc.collect()` 后全 9e-7 收敛。

2.8 A8 实际输出情况

上述实测下, 输出 x 满足 $\|Dx - b\| < \text{tol}$, 且 `convergence_history` 单调下降 (粗校正前后双采样, `_multigrid.py:434,484`)。未验证: 本次会话未亲自产生输出数值。

2.9 A9 结果分析

结合 A7 文档数据: 多层 (2L) 在 8x8x8x16 上残差 7.8e-7, 说明粗网格校正有效消除了 BiCGStab 单独难以收敛的低频模态; 自动重启 (R2) 使偶发 breakdown 不再中断求解, 收敛不受影响; R3 fix 避免了粗校正后 `rho=vdot(r_tilde_old,r_new)` 无意义导致的收敛退化/发散 (`_multigrid.py:468-473` 注释)。多线程实例并行要求显式资源管理, 否则隐式 GC 与 C++ 操作并发导致残差 $O(1)$ (`_multigrid.py:290-303`)。

2.10 A10 综合分析

V-cycle 是 PyQCU 与朴素 BiCGStab 的根本分野: 它将「红外慢模态」显式下推到粗网格用更少自由度求解。Galerkin 投影保证粗算子与细算子谱一致 (`_operator.py:158` 注释); 5-stream 与 device 端标量 (`cpp/.../AGENTS.md` 关键不变量 1) 保证 C++ 平滑的高吞吐; R3/R2 两处 fix 是工程鲁棒性的关键, 缺失其一即退化或发散。这与纯 Python 参考实现 (`_multigrid.py`) 形成「生产后端 + 可验证参考」双轨。

2.11 A11 结尾总结

结论

多重网格 V-cycle = 近零空间 Galerkin 粗化 + 递归粗校正 + BiCGStab 平滑; 其工程正确性依赖三处关键: Galerkin 谱一致性、粗校正后 R3 状态重置、breakdown 自动重启 (R2)。PyQCU 以「C++ CUDA 5-stream 细层 + 纯 Python 粗层」混合, 兼顾性能与可验证性。

2.12 A12 结尾评价

优点: 低频误差消除显著、收敛与体积解耦、双后端可交叉验证、R2/R3 提升鲁棒性。**可改进:** null 向量迭代每向量一个 LatticeSet, 大格子 ($16^3 \times 32$) 单实例 init 仍可能 OOM (`pyqcu/solver/AGENTS.md` 「多实例资源管理」); 粗算子 33-tensor 宽 stencil 构建成本需批量优化 (见 C6)。